- Give the "old" and the "modern" definition of an embedded system. Classify each of the following under the modern definition and justify: (a) a smart thermostat, (b) a gaming laptop, (c) a car's anti-lock braking controller, (d) a smartwatch.

- "A real-time system is a fast system." Is this this statement is valid? Elaborate and justify your answer in one or two sentences..

- Classify each of the following as hard or soft real-time and justify your answer by describing what happens when a deadline is missed: (a) video conferencing, (b) airbag, (c) a pacemaker, (d) an online music streaming app, (e) a factory robot arm that must stop when a person enters its zone.

- What is cross-compilation? Why is it necessary when programming a microcontroller such as the MSP432?

- Name the three main components of a basic computer system and the three groups of signals carried on the system bus. For each bus group, state what it carries.

- Describe the role of each of these registers: PC, IR, MAR, SR, SP.

- A processor executes a memory WRITE instruction. (a) Draw a timing diagram with the clock, address bus, data bus, READ and WRITE signals over two clock cycles. (5) (b) Explain why the fetch–execute sequence must be performed in order and what triggers each step. (3)

- Compare the timing of a memory READ instruction with a memory WRITE instruction after the fetch phase. What happens on the second rising edge and the second falling edge in each case?

- A simple processor uses the 2-bit opcode table `ADD=00, SUB=01, MOV=10, INPUT=11`. How many distinct operations can this instruction set encode? If the designer wants to add 5 more operations, what is the minimum opcode width required?

- In the ARM Cortex-M register bank, identify R13, R14 and R15 and describe the function of each. What happens to the PC and LR when a function is called and when it returns?

- Two 32-bit values are added on an ARM Cortex-M. Give the values of the N, Z, C and V flags in the APSR for each case: (a) `0x7FFFFFFF + 0x00000001`, (b) `0xFFFFFFFF + 0x00000001`, (c) `0x00000005 + 0x00000003`.

- Given `#define SQUARE(x) x*x`, what is the value of y after `int a = 3; int y = SQUARE(a + 1);`? Explain the result and rewrite the macro correctly if needed.

- Let `x = 2` and `y = 1`. Evaluate `x && y`, `x || y`, `x & y` and `x | y`. Why is confusing & and && a common source of bugs when testing register bits?

- Evaluate `(uint8_t)(~((1<<5) | (1<<2)))` step by step, showing each intermediate binary value. What operation would you use with this mask, and what does it accomplish?

- Why is `int` "not portable," and why does the course require types like `uint8_t` and `uint32_t`? Give a concrete scenario where using the wrong size would break code that accesses a peripheral register.

- What problem do include guards (`#ifndef / #define / #endif`) solve? Write a correct header file `led.h` that declares (but does not define) `void led_toggle(uint8_t pin);`.

- What does each of the following return over five successive calls? Explain.

```
int f(void) { int x = 0; x++; return x; }
int g(void) { static int x = 0; x++; return x; }
```

- Consider:

```
int i = 0;
while (i == 0) { }
```

where `i` is changed by an ISR. What might an optimizing compiler do with this loop, and how does `volatile` fix the problem?

- What is the size, in bytes, of each type below? Explain the difference between `struct` and `union` in terms of memory layout.

```
struct S { uint32_t a; uint32_t b; };
union  U { uint32_t a; uint8_t bytes[4]; };
```

- Running `int *p = (int *)100000; *p = 0;` on a Windows or Linux PC will likely crash, but on the MSP432 it may "work." Explain why.

- Compare isolated I/O and memory-mapped I/O. Which scheme does the ARM Cortex-M use, and what practical consequence does that have for C programmers?

- Compare polling and interrupts as methods to synchronize the CPU with an interface chip. Give one advantage of each.

- For a GPIO pin, the registers `PxSEL0` and `PxSEL1` select the pin function. Fill in the table for all four combinations. Why must your code explicitly clear these registers even though they default to 0 on reset?

- Write the complete C initialization code for: a push-button on P3.5 that is active high (needs a pull-down resistor) and an active-high LED on P2.4 that should start OFF. Use shift notation and `uint8_t` casts.

- Explain switch bounce and describe the three-step debouncing algorithm from the notes.

- What is a watchdog timer, how does it work, and why does the first line of every `main` in this course stop it? What would happen if we forgot to stop it and never serviced it?